Instituto Federal de Educação, Ciência e Tecnologia
do Rio Grande do Norte

Ian de Araújo, Jardson Alan

Agrupamento de dados sobre redes sociais e saúde mental de estudantes

Rio Grande do Norte
2026

Ian de Araújo, Jardson Alan

Agrupamento de dados sobre redes sociais e saúde mental de estudantes

Artigo apresentado à disciplina de Análise de Dados do Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Norte, com o registro da etapa de agrupamento do projeto.

Rio Grande do Norte
2026

Agrupamento de dados sobre redes sociais e saúde mental de estudantes

Ian de Araújo[1] e Jardson Alan[1]

RESUMO

Esta etapa agrupa 4.500 estudantes com sete medidas numéricas: idade, horas diárias de redes sociais, horas extras no fim de semana, duração e qualidade do sono, estresse percebido e GPA. O índice de saúde mental permanece fora das entradas, por ser o atributo-alvo do estudo. Os vazios de estresse e de GPA recebem a média truncada por idade, e as colunas são padronizadas. São aplicados o K-Means e o agrupamento hierárquico com ligação de Ward. O número de grupos é o valor de k, entre 2 e 6, com maior *Silhouette Score* no K-Means. Esse valor é 2, e o mesmo k é usado no algoritmo hierárquico. Nesse corte, o K-Means supera o hierárquico nas métricas internas e na concordância com a rotulação `Overall_Impact`. O grupo de maior uso diário reúne todos os registros `Negative` e quase todos os `Neutral`. O outro grupo é formado sobretudo por registros `Beneficial`.

Palavras-chave: Agrupamento. K-Means. Agrupamento hierárquico. Redes sociais. Saúde mental.

1 ETAPA DE AGRUPAMENTO

O registro desta etapa é de 1 de novembro de 2026. A implementação está em `src/clustering/grouping.py` e é chamada por `main.py` depois do pré-processamento. O conjunto continua sendo `data/raw/dataset.csv`, com 4.500 estudantes (YADAV, 2026).

O objetivo desta entrega é formar grupos com mais de um algoritmo e comparar os grupos descobertos com uma rotulação já existente no arquivo. A rotulação usada na comparação é `Overall_Impact`, com as classes `Beneficial`, `Neutral` e `Negative`. Ela não entra como variável dos algoritmos. `Mental_Health_Index` também fica de fora das entradas, porque é o atributo-alvo do estudo e não deve definir os grupos que depois seriam descritos por ele.

2 VARIÁVEIS

As colunas usadas para formar os grupos são as sete abaixo. `Student_ID` não participa, porque é apenas um identificador.

- `Age`, `Daily_Usage_Hours` e `Weekend_Extra_Hours`.
- `Sleep_Duration_Hours` e `Sleep_Quality_Score`.
- `Perceived_Stress_Score` e `Academic_Performance_GPA`.

[1] Estudantes da disciplina de Análise de Dados do Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Norte.

Os valores ausentes de estresse e de GPA são preenchidos com a média truncada de 10% em cada extremidade, dentro da mesma idade. Em seguida, as sete colunas passam por `StandardScaler`, para que uma escala maior não domine o cálculo das distâncias. Os algoritmos e as métricas vêm do scikit-learn.

3 ALGORITMOS

São aplicados dois algoritmos.

O K-Means é particional e representa cada grupo por um centroide. Na busca do número de grupos, k varia de 2 a 6, com inicialização `k-means++`, 20 inicializações e `random_state` igual a 42. O k escolhido é o de maior *Silhouette Score*. O modelo final repete o K-Means com esse k, 50 inicializações e no máximo 500 iterações.

O segundo algoritmo é o `AgglomerativeClustering`, hierárquico, com ligação de Ward e o mesmo número de grupos do K-Means. A ligação de Ward usa distância euclidiana, compatível com as colunas já padronizadas.

4 ESCOLHA DO NÚMERO DE GRUPOS

A Tabela 1 reúne as métricas do K-Means para cada k. Silhouette e Calinski-Harabasz são maiores quando os grupos estão mais separados. Davies-Bouldin é menor nesse caso. O *Adjusted Rand Index* (ARI) e a *Normalized Mutual Information* (NMI) comparam os grupos com `Overall_Impact`: quanto mais próximos de 1, maior a concordância com essa rotulação.

Tabela 1 – Métricas do K-Means para k de 2 a 6

k	Silhouette	Calinski-Harabasz	Davies-Bouldin	ARI	NMI
2	0,2993	2307,90	1,2769	0,4106	0,3778
3	0,1982	1738,36	1,5967	0,2767	0,3859
4	0,1802	1421,98	1,6247	0,1754	0,3268
5	0,1632	1251,70	1,6373	0,1075	0,2896
6	0,1549	1147,06	1,5516	0,0925	0,2809

Fonte: Elaborado pelos autores (2026), pela execução de `python main.py`.

O maior Silhouette Score é 0,2993, em $k = 2$. Esse corte também tem o maior Calinski-Harabasz, o menor Davies-Bouldin e o maior ARI. A NMI é um pouco maior em $k = 3$ (0,3859 contra 0,3778), mas a regra implementada escolhe o k pela silhueta. Os dois algoritmos seguem, portanto, com dois grupos.

5 COMPARATIVO ENTRE OS ALGORITMOS

A Tabela 2 compara o K-Means final e o agrupamento hierárquico, ambos com dois grupos. O K-Means fica à frente nas três métricas internas: silhueta 0,2993 contra 0,2442, Calinski-Harabasz 2307,90 contra 1829,67 e Davies-Bouldin 1,2769 contra 1,4573. Também fica à frente na concordância com `Overall_Impact`: ARI 0,4106 contra 0,2505 e NMI 0,3778 contra 0,2421.

Tabela 2 – Comparação entre K-Means e agrupamento hierárquico, com $k = 2$

Algoritmo	Silhouette	Calinski-Harabasz	Davies-Bouldin	ARI	NMI
K-Means	0,2993	2307,90	1,2769	0,4106	0,3778
Hierárquico	0,2442	1829,67	1,4573	0,2505	0,2421

Fonte: Elaborado pelos autores (2026), pela execução de `python main.py`.

6 PERFIL DOS GRUPOS

A Tabela 3 traz a média de cada variável no K-Means. A idade e as horas extras de fim de semana quase não separam os grupos. O que os separa é o uso diário, o sono, o estresse e o GPA.

Tabela 3 – Média das variáveis em cada grupo do K-Means

Grupo	Idade	Uso diário	Sono	Qualidade	Estresse	GPA
0	18,64	7,94	5,56	1,48	20,02	3,08
1	18,74	3,92	7,30	2,99	10,12	3,61

Fonte: Elaborado pelos autores (2026), pela execução de `python main.py`.

O grupo 0 reúne estudantes com mais horas diárias de redes sociais (7,94), menos sono (5,56 horas), pior qualidade de sono (1,48), mais estresse (20,02) e GPA mais baixo (3,08). O grupo 1 inverte esse perfil: 3,92 horas diárias, 7,30 horas de sono, qualidade 2,99, estresse 10,12 e GPA 3,61. As horas extras de fim de semana ficam em 1,78 no grupo 0 e 1,80 no grupo 1.

O código calcula esse perfil só para o K-Means.

7 COMPARATIVO COM A ROTULAÇÃO EXISTENTE

`Overall_Impact` tem 3.681 registros `Beneficial`, 654 `Neutral` e 165 `Negative`. As Tabelas 4 e 5 cruzam essa rotulação com os grupos de cada algoritmo. Os números dos grupos são atribuídos por cada algoritmo. Nesta execução, os dois colocaram o mesmo tipo de estudante no grupo 0.

Tabela 4 – Grupos do K-Means e rotulação `Overall_Impact`

`Overall_Impact`	Grupo 0	Grupo 1
Beneficial	728	2953
Negative	165	0
Neutral	648	6

Fonte: Elaborado pelos autores (2026), pela execução de `python main.py`.

Tabela 5 – Grupos do algoritmo hierárquico e rotulação `Overall_Impact`

`Overall_Impact`	Grupo 0	Grupo 1
Beneficial	1002	2679
Negative	165	0
Neutral	600	54

Fonte: Elaborado pelos autores (2026), pela execução de `python main.py`.

Nos dois algoritmos, os 165 registros `Negative` caem no grupo 0, o de maior uso diário e maior estresse no perfil do K-Means. O `Neutral` também se concentra nesse grupo: 648 contra 6 no K-Means e 600 contra 54 no hierárquico. O `Beneficial` predomina no grupo 1: 2.953 contra 728 no K-Means e 2.679 contra 1.002 no hierárquico. A concordância é parcial. Há três classes e dois grupos, e uma parte dos registros `Beneficial` permanece no grupo de maior uso. O ARI de 0,4106 no K-Means, contra 0,2505 no hierárquico, quantifica essa diferença: o K-Means acompanha melhor a rotulação existente, sem reproduzi-la.

8 CÓDIGO UTILIZADO

As listagens abaixo são os arquivos desta etapa, na forma em que estão no repositório. `grouping.py` prepara as sete variáveis, escolhe k, executa os dois algoritmos e monta as tabelas de comparação. `main.py` chama essa rotina depois do pré-processamento e imprime os resultados.

8.1 CHAMADA NO FLUXO

Arquivo `main.py`.

```
1 from src.data.load_data import load_data
2 from src.preprocessing.clean_data import select_numeric, null_values
3 from src.preprocessing.standardize_data import standardize_numeric_data
4 from src.visualization.charts import data_standardization_comparison
5 from src.clustering.grouping import run_grouping
6
7 def main():
```

```
    # Carrega os dados do arquivo CSV
    data = load_data("data/raw/dataset.csv")
    print("Dados carregados com sucesso")

    # Seleciona os dados numéricos
    data_numeric = select_numeric(data)
    print("Dados numéricos selecionados com sucesso")

    # Preenche os valores nulos com a média da coluna (filtrada pela idade)
    data_null = null_values(data_numeric)
    print("Valores nulos preenchidos com sucesso")

    # Padroniza os dados numéricos
    data_scaled = standardize_numeric_data(data_null)
    print("Dados padronizados com sucesso")

    # Compara os dados antes e depois da padronização
    data_standardization_comparison(data, data_scaled)
    print("Comparação dos dados antes e depois da padronização realizada com sucesso
      ")

    # Executa o agrupamento e calcula as métricas
    results, comparison, profile, kmeans_label_comparison,
      hierarchical_label_comparison = run_grouping(data)

    print("\nResultados para escolha de k:")
    print(results)

    print("\nComparação entre algoritmos:")
    print(comparison)

    print("\nPerfil dos clusters:")
    print(profile)

    print("\nK-Means - Clusters versus Overall_Impact:")
    print(kmeans_label_comparison)

    print("\nHierárquico - Clusters versus Overall_Impact:")
    print(hierarchical_label_comparison)

if __name__ == "__main__":
    main()
```

8.2 AGRUPAMENTO

Arquivo `src/clustering/grouping.py`.

```python
import pandas as pd

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import (
  silhouette_score,
  calinski_harabasz_score,
  davies_bouldin_score,
  adjusted_rand_score,
  normalized_mutual_info_score,
)
from sklearn.preprocessing import LabelEncoder

from src.preprocessing.clean_data import null_values
from src.preprocessing.standardize_data import standardize_numeric_data

# Define as colunas que serão utilizadas para o agrupamento
FEATURES = [
  "Age",
  "Daily_Usage_Hours",
  "Weekend_Extra_Hours",
  "Sleep_Duration_Hours",
  "Sleep_Quality_Score",
  "Perceived_Stress_Score",
  "Academic_Performance_GPA",
]

# Prepara os dados para o agrupamento
def prepare_data(data: pd.DataFrame):
  features = data[FEATURES].copy()
  features = null_values(features)
  features_scaled = standardize_numeric_data(features)

  labels = LabelEncoder().fit_transform(
    data["Overall_Impact"]
  )

  return features, features_scaled, labels

# Calcula as métricas de avaliação do agrupamento
def calculate_metrics(data, clusters, labels):
  return {
    "silhouette": silhouette_score(data, clusters),
    "calinski_harabasz": calinski_harabasz_score(
      data,
      clusters,
    ),
    "davies_bouldin": davies_bouldin_score(
```

```
      data,
      clusters,
    ),
    "adjusted_rand": adjusted_rand_score(
      labels,
      clusters,
    ),
    "normalized_mutual_info": normalized_mutual_info_score(
      labels,
      clusters,
    ),
  }

# Executa o agrupamento e calcula as métricas
def run_grouping(data: pd.DataFrame):
  features, features_scaled, labels = prepare_data(data)

  results = []

  for number_of_clusters in range(2, 7):
    kmeans = KMeans(
      n_clusters=number_of_clusters,
      init="k-means++",
      n_init=20,
      random_state=42,
    )

    kmeans_clusters = kmeans.fit_predict(features_scaled)

    results.append({
      "algorithm": "K-Means",
      "k": number_of_clusters,
      **calculate_metrics(
        features_scaled,
        kmeans_clusters,
        labels,
      ),
    })

  results_table = pd.DataFrame(results)

  best_k = int(
    results_table.loc[
      results_table["silhouette"].idxmax(),
      "k",
    ]
  )
```

```
95
96  kmeans = KMeans(
97    n_clusters=best_k,
98    init="k-means++",
99    n_init=50,
100   max_iter=500,
101   random_state=42,
102 )
103
104 kmeans_clusters = kmeans.fit_predict(features_scaled)
105
106 hierarchical = AgglomerativeClustering(
107   n_clusters=best_k,
108   linkage="ward",
109 )
110
111 hierarchical_clusters = hierarchical.fit_predict(
112   features_scaled
113 )
114
115 comparison = pd.DataFrame({
116   "K-Means": calculate_metrics(
117     features_scaled,
118     kmeans_clusters,
119     labels,
120   ),
121   "Hierárquico": calculate_metrics(
122     features_scaled,
123     hierarchical_clusters,
124     labels,
125   ),
126 }).T
127
128 profile = features.copy()
129 profile["KMeans_Cluster"] = kmeans_clusters
130
131 cluster_profile = profile.groupby(
132   "KMeans_Cluster"
133 )[FEATURES].mean()
134
135 kmeans_label_comparison = pd.crosstab(
136   data["Overall_Impact"],
137   kmeans_clusters,
138   rownames=["Overall_Impact"],
139   colnames=["Cluster K-Means"],
140 )
141
```

```
  hierarchical_label_comparison = pd.crosstab(
    data["Overall_Impact"],
    hierarchical_clusters,
    rownames=["Overall_Impact"],
    colnames=["Cluster Hierárquico"],
  )

  return (
    results_table,
    comparison,
    cluster_profile,
    kmeans_label_comparison,
    hierarchical_label_comparison,
  )
```

REFERÊNCIAS

[1] YADAV, Harish. **Impact of Social Media on Life**. 2026. Conjunto de dados. Disponível em: https://www.kaggle.com/datasets/harishyadav0506/impact-of-social-media-on-life. Acesso em: 2 nov. 2026.